RELATÓRIO TÉCNICO CONSOLIDADO

Dispositivo IoT para Monitoramento Preditivo e Proteção de Motores AC

KIIRA Motor Guard

Projeto Final — Residência EmbarcaTech

Plataforma: KIIRA — ESP32-S3 Dual-Core Xtensa LX7
Linguagem: C/C++ (ESP-IDF via PlatformIO)
VSCode + PlatformIO | Autodesk Inventor | Maio de 2026

Autores: Klysmann G. F. Barbosa
Luciano R. de Morais

Sumário

a Apresentação **2**

b Objetivos e Viabilidade **2**

b.1 Objetivo Geral . . . 2

b.2 Objetivos Específicos . . . 2

b.3 Viabilidade Técnica e Operacional . . . 2

c Requisitos do Sistema **3**

c.1 Requisitos Funcionais . . . 3

c.2 Requisitos Não Funcionais . . . 3

d Mapeamento de Hardware e Componentes **4**

d.1 Identificação dos Periféricos . . . 4

d.2 Pinout — Tabela de GPIOs . . . 4

d.3 Barramentos Internos . . . 5

e Diagramas do Sistema **5**

e.1 Diagrama de Blocos . . . 5

e.2 Máquina de Estados . . . 5

f Arquitetura do Firmware **5**

f.1 Limiares de Proteção . . . 6

g Processamento Local de Vibração **6**

h Comunicação IoT e Nuvem **7**

h.1 Protocolos Utilizados . . . 7

h.2 Árvore de Tópicos MQTT . . . 7

h.3 Payloads JSON . . . 7

h.4 Upload de Áudio . . . 8

i Arquitetura de Nuvem e Auditoria **8**

j Requisitos Mecânicos e Modelagem 3D **9**

k Processo de Implementação e Status **9**

l Uso de IA no Projeto **9**

m Conclusão **10**

a) Apresentação

O **KIIRA Motor Guard** é um dispositivo IoT embarcado para monitoramento preditivo e proteção automática de motores elétricos AC em ambiente industrial. O sistema é fixado à carcaça do motor e coleta continuamente dados de vibração, temperatura, umidade e áudio, processando métricas localmente no ESP32-S3 antes de transmitir telemetria para uma infraestrutura de nuvem e dashboard.

A motivação principal é reduzir paradas não programadas em linhas de produção. Falhas em motores AC podem interromper processos inteiros, elevar custos de manutenção corretiva e gerar riscos operacionais. O sistema proposto atua como uma camada de sensoriamento, diagnóstico e proteção, combinando manutenção preditiva com desarme automático em condições críticas.

Dois mecanismos críticos sustentam a operação:

- **Proteção automática por relé**: desarme imediato do motor quando a temperatura ou a vibração ultrapassam limiares críticos, seguido de bloqueio temporário de segurança;
- **Análise acústica remota**: gravação sob demanda de áudio do motor e envio ao servidor para análise especializada de ruídos anormais.

O motor pode ser ligado, desligado, rearmado e configurado remotamente via MQTT, preservando também controles físicos locais para reset e silenciamento do buzzer.

b) Objetivos e Viabilidade

Objetivo Geral

Desenvolver um sistema embarcado de baixo custo, robusto e escalável para monitorar motores AC, detectar sinais de falha iminente e acionar proteção automática antes que danos catastróficos ocorram.

Objetivos Específicos

- Monitorar vibração em três eixos e calcular localmente RMS, pico e resultante tridimensional;
- Monitorar temperatura e umidade associadas à carcaça e ao ambiente do motor;
- Exibir telemetria e estado do motor em display OLED local;
- Publicar telemetria e status em dashboard via MQTT com QoS 1;
- Receber comandos remotos de operação, reconfiguração e captura de áudio;
- Registrar backup local em MicroSD para tolerância a falhas de rede;
- Prover mecanismo de auditoria para eventos críticos e mudanças de parâmetros.

Viabilidade Técnica e Operacional

A solução é tecnicamente viável porque utiliza componentes acessíveis, conhecidos e compatíveis com ambientes industriais. O ESP32-S3 fornece conectividade Wi-Fi integrada, dois núcleos físicos para isolamento de tarefas e desempenho suficiente para cálculo local de métricas. A arquitetura FreeRTOS evita que operações de rede bloqueiem a lógica de proteção, e a comunicação híbrida MQTT + HTTP reduz o tráfego indevido no broker.

Operacionalmente, o sistema reduz inspeções manuais rotineiras, permite priorização de manutenção, aumenta a segurança por desarme automático e viabiliza acompanhamento centralizado de múltiplos motores em uma mesma planta.

c) Requisitos do Sistema

Requisitos Funcionais

Tabela 1: Requisitos funcionais consolidados

ID	Descrição	Prioridade
RF01	Monitorar continuamente aceleração nos eixos X, Y e Z com MPU6050.	Alta
RF02	Calcular localmente vibração RMS, resultante tridimensional e pico para reduzir envio de dados brutos.	Alta
RF03	Monitorar temperatura e umidade com DHT22.	Alta
RF04	Exibir telemetria e estado do motor no OLED local.	Média
RF05	Publicar telemetria e status periodicamente via MQTT.	Alta
RF06	Desligar automaticamente o motor via relé quando limites críticos forem excedidos.	Alta
RF07	Emitir alerta sonoro por buzzer em estados de alerta e proteção.	Média
RF08	Aplicar cooldown de segurança de 60 s após desarme.	Alta
RF09	Permitir religamento por botão local ou comando remoto após cooldown.	Alta
RF10	Silenciar temporariamente o alarme por botão físico.	Baixa
RF11	Receber comandos MQTT para ligar, desligar, resetar proteção e gravar áudio.	Alta
RF12	Gravar áudio de 5 s com microfone digital I2S mediante comando remoto.	Alta
RF13	Enviar áudio gravado via HTTP POST multipart.	Alta
RF14	Salvar backup local em CSV no cartão MicroSD.	Média
RF15	Permitir configuração remota dos limiares de alerta e criticidade.	Média

Requisitos Não Funcionais

Tabela 2: Requisitos não funcionais consolidados

ID	Descrição	Categoria
RNF01	Firmware em C/C++ com ESP-IDF via PlatformIO.	Implementação
RNF02	Uso de FreeRTOS para concorrência entre sensores, proteção, comunicação e interface.	Arquitetura
RNF03	Tarefas críticas de segurança isoladas do fluxo de interface e áudio.	Segurança
RNF04	Comunicação MQTT com QoS 1 e LWT para indicar perda de conexão.	Confiabilidade
RNF05	Tempo de resposta entre anomalia crítica e acionamento do relé inferior a 100 ms.	Desempenho
RNF06	Gabinete com acoplamento mecânico rígido, ventilação, duto acústico e proteção contra poeira.	Mecânica
RNF07	Operação em ambiente com ruído eletromagnético, priorizando microfone digital I2S e relé optoacoplado.	Robustez
RNF08	Escalabilidade para múltiplos motores via árvore de tópicos motor/{id}/...	Escalabilidade
RNF09	Código modular, versionado e documentado.	Manutenção

d) Mapeamento de Hardware e Componentes

Identificação dos Periféricos

Tabela 3: Componentes utilizados no sistema

Componente	Modelo	Interface	Função
Microcontrolador	ESP32-S3 (KIIRA)	—	Processamento dual-core, Wi-Fi, Bluetooth LE e execução do firmware FreeRTOS.
Acelerômetro/Giroscópio	MPU6050	I2C	Medição de vibração em três eixos para detecção de desalinhamento, desgaste e cavitação.
Sensor de temperatura/umidade	DHT22 / AM2302	Single-wire	Monitoramento térmico e umidade ambiente.
Microfone digital	INMP441	I2S	Captura acústica com menor suscetibilidade a ruído analógico.
Display	OLED SSD1306 128x64	I2C	Interface local de diagnóstico.
Relé	Módulo optoacoplado	GPIO	Proteção elétrica do motor por comando de contator.
Buzzer	Ativo / PWM	LEDC	Alertas sonoros locais.
Cartão MicroSD	Módulo SPI	SPI	Backup local em CSV e suporte a arquivos de áudio.
Botões físicos	Táctil	GPIO	Reset de proteção e silenciamento do buzzer.

Pinout — Tabela de GPIOs

Tabela 4: Mapeamento consolidado de GPIOs

Periférico / Barramento	Sinal	GPIO	Observação
I2C0 compartilhado	SDA	GPIO 14	MPU6050 + OLED; pull-up externo recomendado.
I2C0 compartilhado	SCL	GPIO 21	Barramento Fast Mode 400 kHz.
INMP441 / I2S0	SCK / BCLK	GPIO 2	Relógio de bits da captura.
INMP441 / I2S0	WS / LRCLK	GPIO 37	Seleção de canal.
INMP441 / I2S0	SD	GPIO 35	Linha de dados do microfone.
DHT22	DATA	GPIO 36	Single-wire com pull-up de 10 kΩ.
Relé	CONTROL	GPIO 47	Saída digital para proteção do motor.
Buzzer	PWM	GPIO 3	PWM via LEDC.
MicroSD / SPI	CS	GPIO 38	Seleção do cartão.
MicroSD / SPI	SCK	GPIO 40	Clock SPI.
MicroSD / SPI	MOSI	GPIO 39	Dados para o cartão.
MicroSD / SPI	MISO	GPIO 41	Dados do cartão.
Botão Reset	IN	GPIO 46	Entrada digital com debounce.
Botão Silenciar	IN	GPIO 45	Entrada digital com debounce.

Observação: Os botões foram alocados em GPIOs neutros para evitar efeitos indesejados de boot. O relé permanece em GPIO 47, com estado inicial seguro para evitar acionamentos aleatórios.

Barramentos Internos

Tabela 5: Barramentos internos do ESP32-S3

Barramento	Periféricos	Velocidade / Uso
I2C0	MPU6050 + OLED SSD1306	400 kHz; endereços distintos no mesmo barramento.
I2S0 + DMA	Microfone INMP441	16 kHz, 16 bits, mono; captura de áudio.
SPI2	Cartão MicroSD	Até 20 MHz; backup CSV e arquivos WAV.
Single-wire	DHT22	Leitura periódica de temperatura e umidade.
GPIO / LEDC	Relé, buzzer e botões	Atuação, PWM e entradas digitais com debounce.
Wi-Fi	MQTT + HTTP	Telemetria, comandos e upload de áudio.

e) Diagramas do Sistema

Diagrama de Blocos

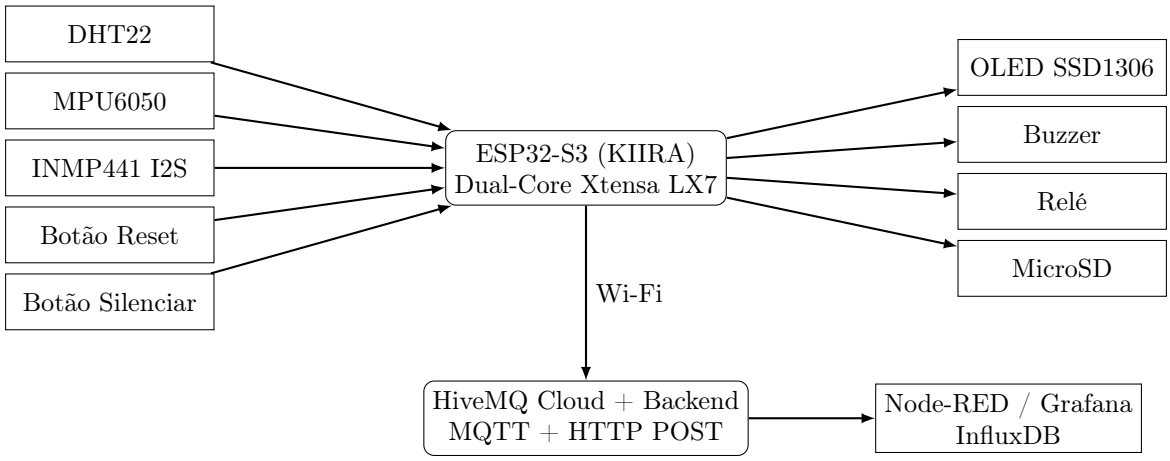


Figura 1: Diagrama de blocos do sistema embarcado

Máquina de Estados

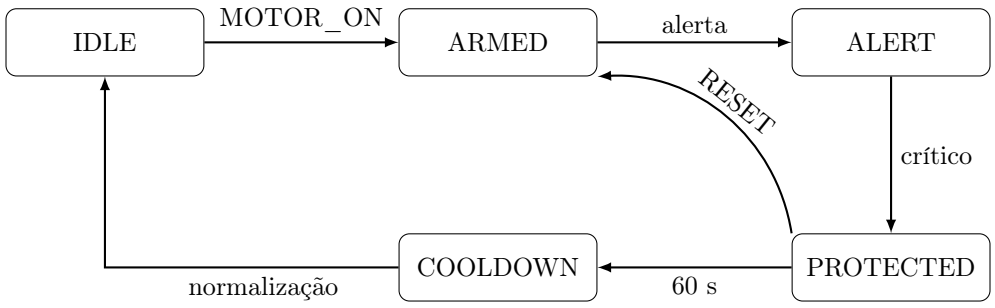


Figura 2: Máquina de estados do motor

f) Arquitetura do Firmware

O firmware foi organizado em tarefas FreeRTOS para isolar rotinas críticas de segurança de operações de latência variável, como Wi-Fi, MQTT, upload HTTP e atualização de interface. A proteção do motor deve continuar funcionando mesmo em caso de instabilidade de rede.

Tabela 6: Divisão de tarefas FreeRTOS

Task	Core	Frequência	Responsabilidade
task_sensors	0	100 ms	Leitura do MPU6050 e DHT22; cálculo local de métricas.
task_protection	0	Alta prioridade	Verificação de limiares e acionamento do relé em menos de 100 ms.
task_telemetry	0	5 s	Publicação MQTT, recepção de comandos e backup em MicroSD.
task_display	1	500 ms	Atualização do OLED sem flicker.
task_buzzer	1	100 ms	Controle sonoro por PWM em alerta e proteção.
task_audio_record	1	Sob demanda	Captura I2S, montagem WAV e envio HTTP POST.

Limiares de Proteção

Tabela 7: Limiares de operação

Parâmetro	Alerta	Crítico
Temperatura	70 °C	85 °C
Vibração RMS composta	1.5 g	3.0 g
Cooldown pós-proteção	—	60 s
Tempo máximo de resposta	—	< 100 ms

g) Processamento Local de Vibração

O ESP32-S3 não envia amostras brutas continuamente para a nuvem. O firmware calcula localmente métricas compactas para preservar largura de banda e reduzir latência. Para cada eixo do MPU6050, o RMS é calculado em uma janela de N amostras:

$$a_{RMS} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(a_i - \bar{a})^2}$$

em que a_i é a aceleração instantânea em g, e $\bar{a}$ remove o offset estático da janela:

$$\bar{a} = \frac{1}{N}\sum_{i=1}^{N} a_i$$

O indicador global de vibração do motor é a resultante tridimensional:

$$A_{comp} = \sqrt{a_{x,RMS}^2 + a_{y,RMS}^2 + a_{z,RMS}^2}$$

Quando $A_{comp} \geq 3.0\,g$ ou a temperatura atinge 85 °C, a `task_protection` abre o relé, muda o estado para `PROTECTED`, publica alerta e inicia o cooldown.

h) Comunicação IoT e Nuvem

Protocolos Utilizados

Tabela 8: Protocolos de comunicação

Canal	Protocolo	Justificativa
Telemetria e comandos	MQTT QoS 1	Baixa latência, comunicação bidirecional e suporte a múltiplos motores.
Status e disponibilidade	MQTT Retained + LWT	Dashboard conhece o estado atual e recebe indicação automática de offline.
Upload de áudio	HTTP POST multipart	Mais adequado para transferência de arquivos WAV de 5 s.
Histórico temporal	InfluxDB	Armazenamento e consulta de séries temporais.
Backup offline	CSV em MicroSD	Tolerância a falhas de rede.

Árvore de Tópicos MQTT

Tabela 9: Tópicos MQTT

Tópico	Direção	Função
motor/{id}/telemetry	Dispositivo → Nuvem	Leituras periódicas de sensores e diagnósticos.
motor/{id}/status	Dispositivo → Nuvem	Estado do motor, conexão, alertas e LWT.
motor/{id}/cmd	Nuvem → Dispositivo	Comandos remotos de operação e testes.
motor/{id}/cmd/response	Dispositivo → Nuvem	Confirmação de execução com `command_id`.
motor/{id}/thresholds	Nuvem → Dispositivo	Reconfiguração remota de limiares.

Payloads JSON

Listing 1: Exemplo de telemetria

```
{
  "timestamp": 1779361642,
  "device_id": "motor_01",
  "sensors": {
    "temp_c": 58.4,
    "humidity_pct": 45.2,
    "vibration": {
      "rms_x_g": 0.42,
      "rms_y_g": 0.35,
      "rms_z_g": 0.78,
      "resultant_rms_g": 0.95,
      "peak_g": 1.45
    }
  },
  "diagnostics": {
    "wifi_rssi_dbm": -65,
    "free_heap_bytes": 184520,
    "uptime_s": 3600
  }
}
```

Listing 2: Exemplo de comando remoto

```
{
  "command_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "action": "TRIGGER_AUDIO_RECORD",
  "params": {
    "duration_s": 5
  }
}
```

Listing 3: Exemplo de alerta crítico

```
{
  "alert": {
    "type": "CRITICAL_SHUTDOWN",
    "cause": "VIBRATION_LIMIT_EXCEEDED",
    "measured_value": 3.42,
    "threshold_limit": 3.00,
    "cooldown_s": 60
  }
}
```

Upload de Áudio

Após receber `TRIGGER_AUDIO_RECORD`, o ESP32-S3 grava 5 segundos de áudio via I2S e envia um arquivo WAV ao servidor central:

`POST http://servidor/api/audio/upload`

A requisição utiliza `multipart/form-data` com os campos `device_id`, `timestamp`, `duration` e `audio`.

i) Arquitetura de Nuvem e Auditoria

Tabela 10: Arquitetura de nuvem

Camada	Ferramenta	Função
Broker MQTT	HiveMQ Cloud	Roteamento de telemetria, status e comandos.
Backend de fluxo	Node-RED	Processamento, regras de automação e integração entre serviços.
Banco temporal	InfluxDB	Histórico de telemetria para manutenção preditiva.
Visualização	Grafana / Dashboard	Monitoramento em tempo real e análise operacional.
Servidor de áudio	Flask ou Node.js	Recebimento e armazenamento de WAV.
Auditoria	Blockchain / Smart Contract	Registro imutável de eventos críticos e mudanças de parâmetros.

Eventos comuns permanecem no banco temporal para eficiência. Eventos críticos, como `CRITICAL_SHUTDOWN`, `BOOT`, `OVERRIDE` e `THRESHOLD_CHANGE`, podem ser registrados em um contrato inteligente de auditoria para garantir rastreabilidade, integridade e transparência em ambientes industriais de missão crítica.

j) Requisitos Mecânicos e Modelagem 3D

Tabela 11: Requisitos mecânicos do gabinete

Requisito	Solução Proposta
Acoplamento mecânico	Base rígida com furos M4, abraçadeiras metálicas ou ímãs de neodímio para transmitir vibração ao MPU6050 sem amortecimento excessivo.
Contato térmico	DHT22 posicionado para medir temperatura do motor sem ser mascarado pelo calor interno do ESP32-S3 e do relé.
Duto acústico	Canal direcionado para o INMP441 com abertura externa protegida contra poeira.
Ventilação	Grelhas laterais ou superiores protegidas por tela/filtro.
Slot MicroSD	Abertura lateral dedicada para manutenção e retirada do cartão.
Janela do display	Encaixe frontal para OLED com proteção mecânica.
Material	ABS, PETG ou Nylon, conforme temperatura e robustez esperadas.

k) Processo de Implementação e Status

O desenvolvimento foi conduzido em fases iterativas: definição de arquitetura e hardware, implementação do firmware FreeRTOS, integração IoT/nuvem e testes de validação.

Tabela 12: Fases de implementação

Fase	Descrição
Arquitetura e hardware	Seleção de sensores, definição de pinagem segura, barramentos internos e requisitos mecânicos.
Firmware	Criação das tasks FreeRTOS, leitura de sensores, cálculo RMS, estados do motor e proteção por relé.
Integração IoT	MQTT com QoS 1, tópicos hierárquicos, HTTP POST de áudio, dashboard e banco temporal.
Validação	Testes de sensores, injeção de falhas, captura I2S, gravação MicroSD e atuação da proteção.

O status consolidado indica dashboard web funcional, injeção de falhas implementada, processamento acústico I2S com fallback de simulação, suporte a MicroSD e fluxo de telemetria integrado. Como próximos passos, recomenda-se avançar em diagnóstico assistido por IA, testes de carga com múltiplos dispositivos e observabilidade avançada com Prometheus e Grafana.

l) Uso de IA no Projeto

O relatório atual foi consolidado a partir do documento técnico original e dos relatórios posteriores da equipe: arquitetura, requisitos, viabilidade empresarial, processo de implementação, arquitetura técnica detalhada e diagramas do sistema. Ferramentas de IA foram utilizadas como apoio à organização textual e revisão estrutural, enquanto validação técnica, decisões de hardware, testes e ajustes finais permanecem sob responsabilidade humana.

Tabela 13: Participação IA × Humano

Componente	IA	Humano
Consolidação e redação	Apoio na organização	Revisão e aprovação
Arquitetura de firmware	Apoio conceitual	Implementação e validação
Mapeamento de hardware	Apoio documental	Conferência em datasheets e placa
Payloads MQTT/HTTP	Apoio de modelagem	Ajustes de negócio e integração
Testes em hardware	0%	100% humano

m) Conclusão

O KIIRA Motor Guard apresenta uma arquitetura embarcada robusta para manutenção preditiva e proteção de motores AC. A combinação de sensores de vibração, temperatura e áudio com processamento local reduz tráfego de rede e permite resposta rápida a condições críticas. A divisão FreeRTOS por tarefas, o uso de MQTT para telemetria e comandos, HTTP para arquivos de áudio, backup local em MicroSD e integração com dashboard tornam a solução tecnicamente consistente e operacionalmente viável.

A solução também se mostra escalável para múltiplos motores por meio de tópicos hierárquicos, parâmetros remotos e infraestrutura de nuvem. Para evolução, recomenda-se priorizar testes em campo, calibração dos limiares por tipo de motor, validação mecânica do gabinete, observabilidade avançada e integração de diagnósticos preditivos assistidos por IA.

Versão consolidada — Maio de 2026